

WT Finance

Briefing de Desenvolvimento

Dashboard de Acompanhamento de Performance — v1

Autor: Yan — Analista Financeiro · Welcome Trips

Versão: 1.0 · Abril de 2026

Status: Pronto para execução pelo Claude Code

0. Como ler este briefing

Este documento é o contrato técnico entre o autor (Yan, product owner) e o Claude Code (executor). Ele descreve, com profundidade, o que será construído, por que, em qual ordem, e com quais critérios de aceite.

0.1 Para quem é este documento

- **Claude Code:** usa este documento como contexto profundo para gerar o código. Lê integralmente antes de qualquer ação. Implementa missão por missão.
- **Yan:** usa como referência permanente do projeto. Consulta para tirar dúvidas sobre decisões já tomadas. Atualiza quando algo muda.
- **Diretoria / futuros mantenedores:** usa para entender o estado da v1 e o porquê das decisões.

0.2 Princípios para o Claude Code seguir

1. **Implementar missão por missão.** Não tentar fazer tudo de uma vez. Cada missão termina com um Pull Request mergeado.
2. **Sempre planejar antes de codar.** Para cada missão, propor um plano para Yan revisar. Só codificar após aprovação.
3. **Respeitar o escopo.** Se algo solicitado não estiver no escopo desta v1 (capítulo 2), avisar Yan e perguntar.
4. **Preferir simplicidade.** Ao escolher entre solução simples e sofisticada, escolher a simples. Esta é a v1, não a versão final.
5. **Documentar decisões relevantes.** Ao tomar uma decisão arquitetural não trivial, criar/atualizar ADR em docs/adr/.
6. **Usar TypeScript estrito.** Sem any, sem ignore. Tipos são contrato.

0.3 Estrutura deste documento

Capítulo	Conteúdo
1	Visão e contexto da v1
2	Escopo (in/out)
3	Arquitetura técnica
4	Modelagem do banco de dados
5	Estratégia de carga de dados
6	APIs (Route Handlers)
7	Componentes do frontend

Capítulo	Conteúdo
8	Tema visual
9	Missões — divisão do trabalho em PRs
10	Premissas e decisões registradas
A	Anexos: números reais, glossário, taxonomia oficial

1. Visão e Contexto

1.1 O que estamos construindo

O primeiro dashboard analítico da plataforma WT Finance. Substitui (em paralelo, ainda não em produção total) o dashboard atual de Power BI da Welcome Trips, e estabelece a base técnica para todas as funcionalidades subsequentes da plataforma.

1.2 Por que estamos construindo

O dashboard atual (Power BI alimentado por RPA + Excel) tem três limitações que justificam a substituição:

- **Fragilidade.** Cadeia RPA → Excel → Power BI tem ponto único de falha (PC dedicado).
- **Pobreza analítica.** Mostra valor realizado vs meta, sem contexto (projeção, gap, ritmo, comparativos).
- **Falta de fundação.** Não estabelece base para crescer (autenticação, permissões, histórico, módulos administrativos).

Esta v1 entrega um dashboard equivalente ao atual, com ganhos analíticos perceptíveis, e estabelece a base de banco de dados, APIs e frontend que será reutilizada nas Ondas 2, 3, 4 e seguintes do projeto.

1.3 Quem usa esta v1

Apenas o autor (Yan) durante a fase de validação. Sem login. Compartilhamento amplo virá em iterações futuras.

1.4 Onde fica

Deploy automático no Vercel, em URL gerada (ex: wt-finance.vercel.app). Dados no Supabase (projeto provisionado em São Paulo, plano free).

1.5 Resultado esperado

Ao final desta v1, o autor abre uma URL e vê:

- Header com KPIs do mês corrente: valor realizado, atingimento %, projeção de fechamento, gap em R\$.
- 4 gauges (geral + 3 setores macro) mostrando progresso vs meta.
- Gráfico de ritmo diário (linha ideal vs realizado acumulado).
- Histórico mensal empilhado por setor (12 meses do ano).
- Top 10 vendedores do mês.
- Top 10 produtos do mês (mix de receita).

- Comparativo YoY: ano vs ano anterior, mês a mês.
- Filtros globais: período (ano/mês) e setor macro.

2. Escopo

2.1 Está dentro do escopo (in)

Item	Descrição
Modelagem do banco	Schemas raw, analytics, app, audit. Tabelas dimensão e fato. RLS configurado.
Carga inicial de dados	Script seed.ts que carrega as 3 planilhas (2024, 2025, 2026) no Supabase.
Carga de metas	Tabela meta_setor populada com metas reais 2026 e fictícias 2024/2025 (decréscimo de 15% a.a.).
APIs	Route Handlers Next.js para servir dados ao frontend.
Dashboard (frontend)	Página principal com 8 componentes especificados.
Filtros	Período (ano/mês) e setor macro.
Tema claro/escuro	Toggle no header. Padrão shadcn/ui.
Deploy automático	Push em main → deploy no Vercel automaticamente.
Documentação técnica	ADRs registrando decisões. README atualizado.

2.2 Não está no escopo (out)

Os itens abaixo são reconhecidamente úteis, mas ficam fora desta v1 para garantir entrega. Aparecerão em iterações futuras.

Item	Será endereçado em...
Login / autenticação	Iteração imediatamente posterior à v1
Cadastro de metas via UI	Onda 2 do roadmap (~2 semanas)
Drill-down por Setor Micro	Iteração futura conforme demanda
Heatmap de sazonalidade	Onda 4 (Análises temporais)
Dashboard de rentabilidade (margem)	Onda 3 do roadmap (~2-3 semanas)
Alertas configuráveis	Onda 5+ (Expansão)
Notificações por e-mail/Slack	Onda 5+
Adaptação do RPA do Power Automate	Próxima iteração após v1
Integração com Monde	Onda 0 estendida — depois desta v1
Versão para TV / modo apresentação	Iteração futura

2.3 Restrições e premissas

Carga única. Esta v1 carrega os dados das planilhas uma única vez (script seed). Não automatiza atualização contínua. Quando dados mudarem, o seed é rodado novamente. Automação real virá com a integração do RPA, fora desta v1.

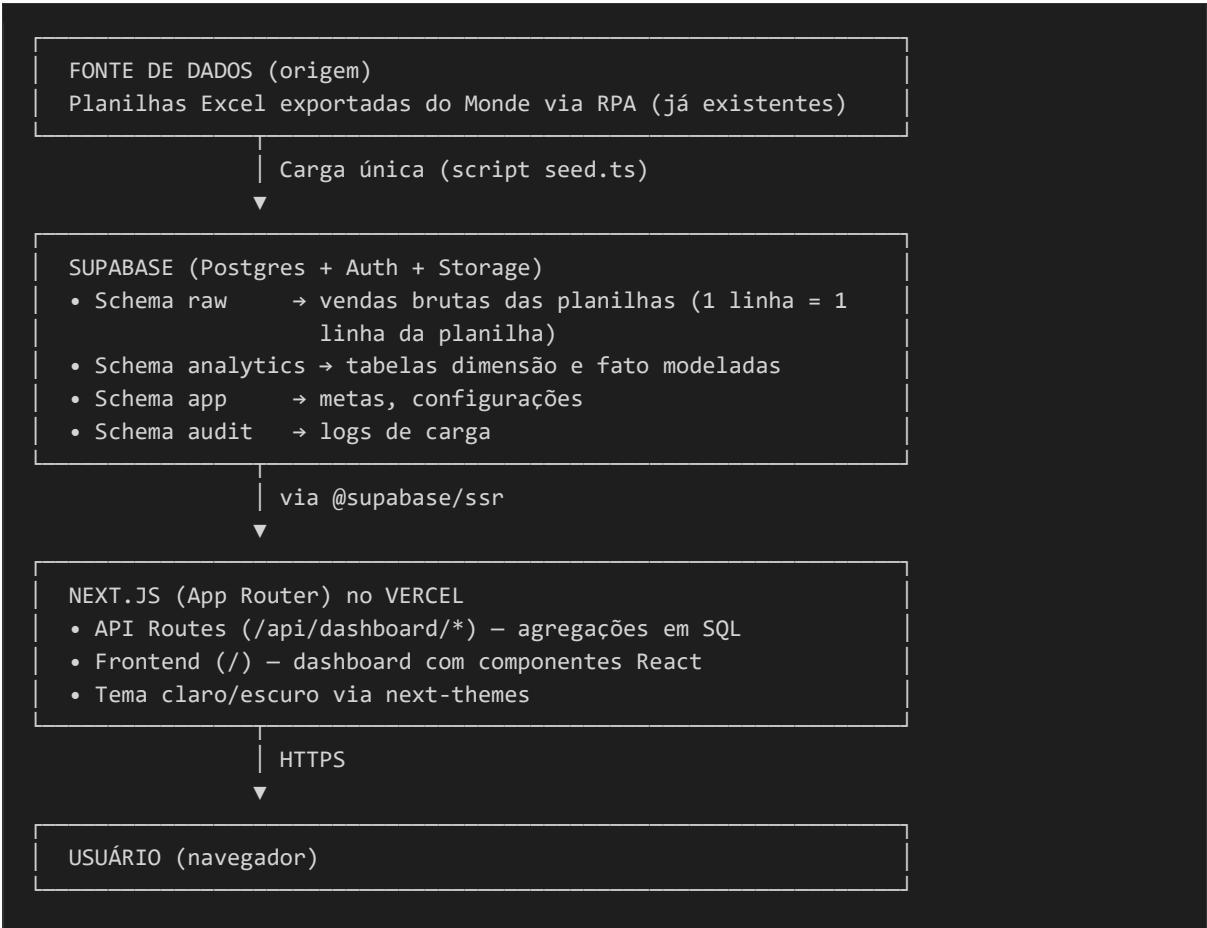
Sem autenticação. URL pública no Vercel, mas obscura. Yan se compromete a não compartilhar a URL fora do uso interno até autenticação ser adicionada.

Metas 2024 e 2025 são fictícias. Calculadas a partir de 2026 com decréscimo de 15% ao ano, preservando a sazonalidade mensal proporcional. Estão claramente sinalizadas como fictícias na tabela de metas e no dashboard (aviso pequeno).

3. Arquitetura Técnica

3.1 Visão geral em camadas

A v1 implementa, em escala reduzida, a arquitetura descrita no Documento de Visão e Arquitetura do projeto:



3.2 Stack confirmada

Camada	Tecnologia	Versão
Banco	Supabase (Postgres 15+)	Plano Free, região São Paulo
Backend	Next.js Route Handlers	Next.js 15+ (App Router)
Frontend	Next.js + React + TypeScript	Next.js 15+, React 19+, TS estrito
UI	shadcn/ui + Tailwind CSS	Última versão estável
Gráficos	Recharts	v2.x
Tema	next-themes	v0.4.x
Cliente Supabase	@supabase/supabase-js + @supabase/ssr	Última estável

Camada	Tecnologia	Versão
Validação de dados	zod	v3.x — para tipos de API e ingestão
Hospedagem	Vercel	Plano Hobby (free)
Versionamento	Git + GitHub	Repositório privado

3.3 Estrutura de pastas

O projeto Next.js criado pelo create-next-app já tem estrutura base. Esta v1 acrescenta:

```

wt-finance/
├── src/
│   ├── app/
│   │   ├── (dashboard)/          # Rota principal – dashboard
│   │   │   ├── page.tsx
│   │   │   └── layout.tsx
│   │   ├── api/
│   │   │   ├── dashboard/
│   │   │   │   ├── kpis/route.ts
│   │   │   │   ├── ritmo-diario/route.ts
│   │   │   │   ├── historico-mensal/route.ts
│   │   │   │   ├── ranking-vendedores/route.ts
│   │   │   │   ├── ranking-produtos/route.ts
│   │   │   │   └── yoy/route.ts
│   │   ├── layout.tsx           # Tema, providers globais
│   │   └── globals.css
│   ├── components/
│   │   ├── ui/                  # shadcn/ui (instalado por CLI)
│   │   ├── dashboard/
│   │   │   ├── kpi-header.tsx
│   │   │   ├── gauges-row.tsx
│   │   │   ├── ritmo-diario-chart.tsx
│   │   │   ├── historico-mensal-chart.tsx
│   │   │   ├── ranking-vendedores-table.tsx
│   │   │   ├── ranking-produtos-table.tsx
│   │   │   ├── yoy-chart.tsx
│   │   │   └── filters.tsx
│   │   └── theme-toggle.tsx
│   ├── lib/
│   │   ├── supabase/
│   │   │   ├── client.ts        # cliente browser
│   │   │   ├── server.ts        # cliente server
│   │   │   └── admin.ts         # cliente service_role (só seed)
│   │   ├── formatters.ts        # formatBRL, formatPct, etc
│   │   ├── types.ts             # tipos compartilhados
│   │   └── queries/             # SQL queries reutilizáveis
│   ├── types/
│   │   └── database.ts          # tipos gerados do schema Supabase
│   └── supabase/
│       ├── migrations/
│       │   └── 0001_init_schemas.sql

```

```

├── 0002_dimensions.sql
├── 0003_facts.sql
├── 0004_app_tables.sql
├── 0005_audit_tables.sql
├── 0006_views.sql      # views materializadas
├── 0007_rls_policies.sql
├── seed/
│   ├── seed.ts        # script principal de carga
│   ├── parse-excel.ts  # leitor das planilhas
│   ├── load-metas.ts  # carga das metas
│   └── data/           # planilhas .xlsx (gitignored)
├── docs/
│   ├── adr/
│   │   └── 0001-stack-inicial.md
│   └── briefings/
│       └── dashboard-v1-briefing.md # cópia md deste documento
├── public/
├── .env.local          # gitignored
├── .env.example        # template para outros devs
└── ...

```

3.4 Decisões arquiteturais relevantes

Decisões abaixo devem ter ADR criado (ou atualizado) ao longo da execução:

- **Server Components por padrão.** Páginas e componentes são Server Components, exceto onde há interatividade (filtros, gráficos com tooltip). Reduz JS no cliente, melhora SEO e performance.
- **Queries SQL diretas.** Para esta v1, queries são escritas em SQL puro (em `src/lib/queries/`) e executadas via `supabase.rpc()` ou `supabase.from()`. Sem ORM. Mantém simplicidade e fica próximo ao banco.
- **Sem React Server Actions na v1.** Para esta v1 só há leitura de dados. Server Actions virão quando houver formulários (Onda 2).
- **Filtros via Search Params.** Filtros (ano, mês, setor) ficam na URL como query params. Permite copiar/compartilhar URLs com filtros aplicados, e funciona com Server Components.
- **Cache via React Query? Não nesta v1.** Cache do Next.js (`revalidatePath`) e do Supabase já são suficientes. Adicionar React Query só se houver problema mensurável.

4. Modelagem do Banco de Dados

Este é o capítulo mais crítico. Decisões aqui são as mais difíceis de reverter depois. O modelo abaixo é minimamente suficiente para a v1, mas projetado para suportar o roadmap completo.

4.1 Schemas

Schema	Propósito	Quem grava
raw	Espelho dos dados de origem (planilhas), tal como chegam.	Script seed (ingestão). Nunca a aplicação.
analytics	Tabelas fato e dimensão, prontas para consumo. Onde o frontend lê.	Triggers/funções que transformam raw → analytics.
app	Dados que nascem na plataforma (metas, configurações).	Aplicação web. Manualmente nesta v1.
audit	Logs de carga e operações.	Triggers automáticos.

4.2 Schema raw

Espelho fiel das planilhas. Um schema raw para cada origem (por enquanto só Excel; quando o RPA for adaptado, vira schema raw_monde, etc.).

Tabela raw.vendas_excel

Reflete exatamente as colunas das planilhas (exceto Intermediário, descontinuado):

Coluna	Tipo	Origem (Excel)
id	bigserial PK	(gerado)
arquivo_origem	text NOT NULL	(metadado: nome do arquivo)
linha_origem	int NOT NULL	(metadado: número da linha)
carregado_em	timestampz DEFAULT now()	(metadado: timestamp da carga)
venda_numero	text	Venda Nº
data_venda	date	Data Venda
vendedor	text	Vendedor
pagante	text	Pagante
setor	text	Setor
produto	text	Produto
receitas	numeric(14,2)	Receitas
valor_total	numeric(14,2)	Valor Total
semana	int	Semana

Coluna	Tipo	Origem (Excel)
setor_macro	text	Setor Macro
mes	text	Mes
setor_micro	text	Setor Micro
contrato	boolean	Contrato (0/1 → false/true)
taxa_servico	boolean	Taxa de Serviço (0/1 → false/true)

Por que manter raw? Permite reprocessar a transformação sem precisar reler os arquivos Excel. Se descobrirmos amanhã uma regra nova de negócio, apenas re-rodamos a transformação raw → analytics. Sem raw, perdemos a fonte original e dependemos de manter os Excel disponíveis para sempre.

4.3 Schema analytics — Dimensões

Dimensões representam as entidades estáveis do negócio. Cada uma com chave primária artificial (surrogate key, bigserial) e chave natural (texto).

analytics.dim_setor_macro

Coluna	Tipo	Notas
id	bigserial PK	
nome	text NOT NULL UNIQUE	'Lazer', 'Corporativo', 'Weddings'
display_nome	text NOT NULL	Para exibição no dashboard. Ex: 'Trips' para Lazer.
cor_hex	text NOT NULL	Cor de identidade. #378ADD (Lazer), #0F6E56 (Corp), #BA7517 (Wed).
ordem	int NOT NULL	Ordem de exibição. 1=Lazer, 2=Weddings, 3=Corporativo.

Valores fixos (3 linhas). Inseridos via migration.

analytics.dim_setor

Coluna	Tipo	Notas
id	bigserial PK	
nome	text NOT NULL UNIQUE	Ex: 'Lazer', 'Expedições', 'WedMe'
setor_macro_id	bigint NOT NULL FK → dim_setor_macro	

analytics.dim_setor_micro

Coluna	Tipo	Notas
id	bigserial PK	

Coluna	Tipo	Notas
nome	text NOT NULL UNIQUE	Ex: 'Hospedagem', 'Extras', 'WedMe'
setor_id	bigint NOT NULL FK → dim_setor	

analytics.dim_vendedor

Coluna	Tipo	Notas
id	bigserial PK	
nome	text NOT NULL UNIQUE	Nome como aparece no Excel
ativo	boolean DEFAULT true	Para esta v1, todos true. Filtro vem em iterações futuras.
criado_em	timestampz DEFAULT now()	

analytics.dim_pagante

Coluna	Tipo	Notas
id	bigserial PK	
nome	text NOT NULL UNIQUE	
criado_em	timestampz DEFAULT now()	

analytics.dim_produto

Coluna	Tipo	Notas
id	bigserial PK	
nome	text NOT NULL UNIQUE	
categoria	text	Por enquanto null. Pode ser populado em iteração futura para agrupar produtos.

analytics.dim_data

Tabela de calendário. Essencial para análises temporais. Pré-populada via migration com todas as datas de 2024-01-01 até 2030-12-31.

Coluna	Tipo	Notas
data	date PK	
ano	int NOT NULL	
mes	int NOT NULL	1-12
mes_nome	text NOT NULL	'Janeiro', 'Fevereiro', etc.
mes_abrev	text NOT NULL	'jan', 'fev', etc.

Coluna	Tipo	Notas
dia	int NOT NULL	1-31
dia_semana	int NOT NULL	0=domingo, 6=sábado
dia_semana_nome	text NOT NULL	'Segunda', 'Terça', etc.
semana_iso	int NOT NULL	
dia_util	boolean NOT NULL	false em sábados/domingos. Feriados ficam para iteração futura.
dia_util_mes	int	Número do dia útil dentro do mês. Importante para ritmo diário.
dias_uteis_no_mes	int	Total de dias úteis do mês. Importante para projeção.
trimestre	int NOT NULL	1-4

4.4 Schema analytics — Fatos

analytics.fato_venda

Cabeçalho da venda. Uma linha por Venda Nº único.

Coluna	Tipo	Notas
id	bigserial PK	
venda_numero	text NOT NULL UNIQUE	Chave natural
data_venda	date NOT NULL FK → dim_data	
vendedor_id	bigint NOT NULL FK → dim_vendedor	
pagante_id	bigint FK → dim_pagante	Pode ser null
contrato	boolean DEFAULT false	
taxa_servico	boolean DEFAULT false	
criado_em	timestampz DEFAULT now()	

Índices: data_venda, vendedor_id, pagante_id.

analytics.fato_venda_item

Detalhe da venda. Uma linha por linha das planilhas (cada produto da venda).

Coluna	Tipo	Notas
id	bigserial PK	
fato_venda_id	bigint NOT NULL FK → fato_venda	
produto_id	bigint NOT NULL FK → dim_produto	

Coluna	Tipo	Notas
setor_id	bigint NOT NULL FK → dim_setor	Pode divergir do setor do vendedor (ex: vendedor de Lazer vendendo produto de Weddings).
setor_micro_id	bigint NOT NULL FK → dim_setor_micro	
valor_total	numeric(14,2) NOT NULL	Valor bruto da venda do item
receitas	numeric(14,2) NOT NULL	Margem real da agência (pode ser negativa)

Índices compostos: (data_venda da fato_venda, setor_id), (produto_id), (setor_id, setor_micro_id).

Decisão importante: setor por item, não por venda. Em uma mesma venda, pode haver itens de setores diferentes (ex: pacote Trips com extra de Weddings). Por isso, setor é dimensão do item, não do cabeçalho. Soma agregada de Valor Total por setor exige sempre passar por fato_venda_item.

4.5 Schema app

app.meta_setor

Metas mensais por setor macro. Para esta v1, populadas manualmente via SQL/seed. A partir da Onda 2, tela de cadastro virá.

Coluna	Tipo	Notas
id	bigserial PK	
setor_macro_id	bigint NOT NULL FK → dim_setor_macro	
ano	int NOT NULL	
mes	int NOT NULL	1-12
valor_meta	numeric(14,2) NOT NULL	Meta de Valor Total para o mês
fonte	text NOT NULL	'real' (vinda de gestor) ou 'ficticia' (calculada). v1 tem ambas.
criado_em	timestampz DEFAULT now()	
UNIQUE (setor_macro_id, ano, mes)		

app.meta_setor_historico

Para auditoria: snapshot de cada alteração de meta. v1 cria a tabela vazia (será usada na Onda 2).

Estrutura: mesmo schema de meta_setor + colunas: alterado_em, alterado_por, valor_anterior, motivo_alteracao.

4.6 Schema audit

audit.ingestao_log

Log de cada execução de carga (script seed).

Coluna	Tipo	Notas
id	bigserial PK	
fonte	text NOT NULL	'excel-2024', 'excel-2025', 'excel-2026', 'metas'
iniciado_em	timestampz NOT NULL	
finalizado_em	timestampz	
status	text NOT NULL	'sucesso', 'falha', 'em_progresso'
registros_processados	int	
erro_mensagem	text	Em caso de falha

4.7 Views materializadas

Para acelerar consultas do dashboard, criar views materializadas pré-agregadas:

- **analytics.mv_vendas_diarias:** agregação por data + setor_macro_id, com SUM(valor_total), SUM(receitas), COUNT(distinct venda_id).
- **analytics.mv_vendas_mensais:** agregação por ano + mes + setor_macro_id.
- **analytics.mv_ranking_vendedores_mensal:** agregação por ano + mes + vendedor_id + setor_macro_id.
- **analytics.mv_ranking_produtos_mensal:** agregação por ano + mes + produto_id + setor_macro_id.

Views são atualizadas via REFRESH MATERIALIZED VIEW após cada carga (no script seed). Em iterações futuras, com RPA contínuo, vira refresh agendado.

4.8 RLS — Row-Level Security

Para esta v1 sem login, RLS é configurado de forma permissiva, mas a estrutura está pronta para evoluir:

- **Schemas raw e audit:** acesso negado por padrão. Apenas service_role lê/escreve.
- **Schema analytics:** read habilitado para o role anon (frontend). Write apenas service_role.

- **Schema app:** read habilitado para anon. Write apenas service_role nesta v1 (Onda 2 vai diferenciar por papel).

Atenção: 'permissivo' aqui significa 'leitura aberta para o frontend'. Isso é aceitável porque (a) não há dados pessoais sensíveis, (b) a URL não é pública/divulgada, (c) RLS estará configurado e ativo — só com políticas relaxadas. Quando login for adicionado, as políticas evoluem para diferenciar usuários sem reescrita.

5. Estratégia de Carga de Dados

5.1 Visão geral do fluxo

O script seed.ts executa em uma única invocação a carga completa:

7. Lê as 3 planilhas em ./supabase/seed/data/ (gitignored).
8. Para cada planilha, insere todas as linhas em raw.vendas_excel (com arquivo_origem e linha_origem).
9. Roda função SQL transform_raw_to_analytics() que: (a) deduplica e popula dimensões, (b) popula fato_venda e fato_venda_item, (c) registra em audit.ingestao_log.
10. Carrega tabela app.meta_setor com metas reais 2026 (do código) e fictícias 2024/2025 (calculadas).
11. Roda REFRESH MATERIALIZED VIEW para todas as views materializadas.

5.2 Como o script é executado

Comando único, rodado localmente pelo Yan:

```
npm run seed
```

Este comando estará configurado em package.json. Internamente roda via tsx (TypeScript runtime). Usa a service_role key (de .env.local) — apenas localmente, nunca no Vercel.

5.3 Idempotência

O seed deve ser idempotente: rodar várias vezes sem duplicar dados. Estratégia:

12. Antes de ingerir, TRUNCATE em todas as tabelas dos schemas raw, analytics, app (preserva audit).
13. Re-popula tudo do zero.
14. Esta abordagem é simples e segura para v1 (40k linhas). Quando a base crescer ou tiver RPA contínuo, evolui para upsert.

Decisão registrada em ADR. Truncate-and-reload é aceitável aqui porque: (a) volume pequeno, (b) carga única manual nesta v1, (c) audit.ingestao_log preserva histórico de execuções. ADR-0002 deve ser criado documentando isso.

5.4 Mapeamento Excel → raw

Conversões aplicadas pelo parse-excel.ts:

Coluna Excel	Coluna raw	Conversão
Venda Nº	venda_numero	string trim
Data Venda	data_venda	Excel date → ISO date
Vendedor	vendedor	string trim, uppercase para normalização
Pagante	pagante	string trim. Se vazio → NULL
Setor	setor	string trim
Produto	produto	string trim
Receitas	receitas	number. Aceita negativos (prejuízo)
Valor Total	valor_total	number
Semana	semana	int
Setor Macro	setor_macro	Mapeia 'Lazer' → preserva. 'Corporativo' → preserva.
Mes	mes	string (preservar abreviado)
Setor Micro	setor_micro	string trim
Contrato	contrato	0 → false, 1 → true
Taxa de Serviço	taxa_servico	0 → false, 1 → true

5.5 Validações na ingestão

- Linhas com Venda Nº vazio: rejeitar (logar warning).
- Linhas com Data Venda inválida: rejeitar.
- Linhas com Setor Macro fora de ('Lazer', 'Corporativo', 'Weddings'): logar warning, mas aceitar (vai para raw, mas pode falhar na transformação).
- Receitas negativas: aceitar normalmente. Não há regra de validação além disso.

5.6 Carga de metas

As metas são carregadas a partir de uma estrutura TypeScript no próprio código (load-metas.ts):

```
// Metas reais 2026 (vindas da pivot de gestores)
const METAS_2026 = {
  Lazer: { jan: 1730750.02, fev: 2499737.96, /* ... */ },
  Corporativo: { jan: 1268695.00, /* ... */ },
  Weddings: { jan: 2048746.41, /* ... */ },
};

// Metas fictícias: 2025 = 2026 / 1.15, 2024 = 2026 / 1.15²
// Sazonalidade preservada (mesma proporção mensal)
```

Os valores exatos das metas estão no Anexo A.3 deste documento.

6. APIs (Route Handlers)

As APIs servem dados ao frontend. Cada endpoint corresponde a um componente do dashboard.

Padrão de implementação:

- Localização: `src/app/api/dashboard/<endpoint>/route.ts`
- Método: GET com query params
- Retorno: JSON com tipo definido em `src/lib/types.ts`
- Validação de query params: zod
- Cliente Supabase: cliente server-side (`src/lib/supabase/server.ts`)
- Erro: status 400 (validação), 500 (banco), 200 (sucesso)

6.1 Padrão de query params

Filtros globais aceitos por todos os endpoints:

Param	Tipo	Default	Notas
ano	int	ano atual (2026)	Required nos endpoints de período.
mes	int (1-12)	mês atual	Required nos endpoints de mês corrente.
setor_macro	string ou 'todos'	todos	Filtro de setor. 'Lazer' 'Corporativo' 'Weddings' 'todos'.

6.2 GET /api/dashboard/kpis

Propósito: retorna os KPIs do header do dashboard (4 caixas).

Query: `?ano=2026&mes=4&setor_macro=todos`

Resposta:

```
{
  realizado: number,           // valor total acumulado no mês
  meta: number,                // meta do mês (soma se setor='todos')
  atingimento_pct: number,     // (realizado / meta) * 100
  ritmo_ideal_pct: number,    // % esperado dado o dia útil corrente
  projecao_fechamento: number, // realizado / dias_uteis_decorridos *
                                // dias_uteis_no_mes
  gap_reais: number,           // meta - projecao_fechamento
  gap_vendas_estimado: number, // gap_reais / ticket_medio_mes
  ticket_medio_mes: number,
  vendas_count: number
}
```

6.3 GET /api/dashboard/gauges

Propósito: retorna os 4 gauges (geral + 3 setores).

Query: ?ano=2026&mes=4

Resposta:

```
{
  total: { realizado: number, meta: number, atingimento_pct: number,
           projecao: number, gap: number },
  setores: [
    {
      setor_macro: 'Lazer' | 'Corporativo' | 'Weddings',
      display_nome: string,
      cor_hex: string,
      realizado: number,
      meta: number,
      atingimento_pct: number,
      projecao: number,
      gap: number
    },
    // ... 3 itens, ordenados por dim_setor_macro.ordem
  ]
}
```

6.4 GET /api/dashboard/ritmo-diario

Propósito: retorna a série temporal do mês corrente (linha realizado vs ideal).

Query: ?ano=2026&mes=4&setor_macro=todos

Resposta:

```
{
  meta_mes: number,
  dias_uteis_no_mes: number,
  pontos: [
    {
      data: '2026-04-01',
      dia_util_mes: 1,
      realizado_dia: number,           // valor do dia
      realizado_acumulado: number,     // soma até o dia
      ideal_acumulado: number,         // (dia_util_mes / dias_uteis_no_mes) * meta_mes
      eh_dia_util: boolean
    },
    // ... uma entrada por dia do mês
  ]
}
```

6.5 GET /api/dashboard/historico-mensal

Propósito: retorna o histórico mensal empilhado dos 12 meses do ano.

Query: ?ano=2026

Resposta:

```
{
  meses: [
    {
      mes: 1,
      mes_abrev: 'jan',
      lazer: number,
      corporativo: number,
      weddings: number,
      total: number,
      meta_total: number
    },
    // ... 12 entradas
  ]
}
```

6.6 GET /api/dashboard/ranking-vendedores

Propósito: retorna top N vendedores do mês.

Query: ?ano=2026&mes=4&setor_macro=todos&limit=10

Resposta:

```
{
  ranking: [
    {
      vendedor_id: number,
      vendedor_nome: string,
      setor_macro_principal: string, // setor onde mais vende
      valor_total: number,
      receitas: number,
      vendas_count: number,
      ticket_medio: number
    },
    // ... até 10 entradas
  ]
}
```

6.7 GET /api/dashboard/ranking-produtos

Propósito: retorna top N produtos do mês.

Query: ?ano=2026&mes=4&setor_macro=todos&limit=10

Resposta: estrutura espelha ranking-vendedores, com produto_nome, setor_macro_principal, valor_total, receitas, vendas_count.

6.8 GET /api/dashboard/yoy

Propósito: retorna comparativo ano vs ano anterior, mês a mês.

Query: ?ano=2026&setor_macro=todos

Resposta:

```
{
  ano_atual: 2026,
  ano_anterior: 2025,
  meses: [
    {
      mes: 1,
      mes_abrev: 'jan',
      atual: number,      // realizado 2026/jan
      anterior: number,   // realizado 2025/jan
      variacao_pct: number, // ((atual - anterior) / anterior) * 100
      meta_atual: number,
      meta_anterior: number
    },
    // ... 12 entradas. Meses ainda não decorridos no ano atual têm atual=null
  ]
}
```

6.9 Decisões de implementação para todos os endpoints

- **Server Component como consumidor preferencial.** Sempre que possível, a página do dashboard é Server Component e chama as APIs internamente. Reduz fetches do cliente.
- **Use views materializadas onde possível.** Se o endpoint pode ser respondido por uma view materializada existente, use. Senão, query SQL otimizada.
- **Caching.** Adicionar export const revalidate = 300 (5 min) nas Route Handlers que servem dados pesados. Para dados que mudam pouco (histórico anual), revalidate maior.
- **Tratamento de mês corrente.** Se ano/mes for o atual, NÃO cachear pesadamente — cliente pode estar interessado em update mais recente. Mas para meses passados, cache agressivo.

7. Componentes do Frontend

Cada componente é um arquivo TypeScript em `src/components/dashboard/`. Componentes que exibem gráficos são Client Components (usam Recharts, requer 'use client'). Componentes que apenas formatam dados podem ser Server Components.

7.1 Layout geral da página

A página principal do dashboard (`src/app/(dashboard)/page.tsx`) é um Server Component que:

15. Lê os search params (ano, mes, setor_macro) com defaults sensatos.
16. Faz fetches paralelos das APIs necessárias.
17. Renderiza os componentes em ordem visual de cima para baixo.

Estrutura visual:

```
Header: WT Finance · Dashboard · [filtros] · [tema toggle]
KPI Header (4 cards horizontais)
Gauges Row (4 gauges: total + 3 setores)
Ritmo Diário (chart linha) | Histórico Mensal (chart bar)
Top Vendedores (tabela) | Top Produtos (tabela)
YoY Chart (linha 2 séries: ano atual vs anterior)
Footer: 'Metas 2024/2025 são fictícias (cálculo)' (pequeno)
```

7.2 Especificação dos componentes

KpiHeader

- **Arquivo:** `src/components/dashboard/kpi-header.tsx`
- **Tipo:** Server Component
- **Props:** `{ data: KpisResponse }`
- **Visual:** 4 cards em grid horizontal:
 - Card 1: Realizado (valor BRL)
 - Card 2: Atingimento % com indicador colorido (verde $\geq$ ritmo ideal, amarelo até -10pp, vermelho $>$ -10pp)
 - Card 3: Projeção de fechamento (valor BRL)
 - Card 4: Gap em R\$ + estimativa de vendas necessárias

GaugesRow

- **Arquivo:** `src/components/dashboard/gauges-row.tsx`
- **Tipo:** Client Component (Recharts)
- **Props:** `{ data: GaugesResponse }`
- **Visual:** 4 gauges semicirculares:
 - Gauge 1 (maior): Welcome Group total
 - Gauges 2-4: Lazer, Weddings, Corporativo (na ordem do `dim_setor_macro`)
 - Cada gauge mostra: valor realizado, % atingimento, ponteiro vermelho na posição da meta (~100%), valor da meta abaixo
- **Implementação Recharts:** `RadialBarChart` com `domain=[0, 100]` customizado. Considerar componente customizado SVG se Recharts limitar visual.

RitmoDiarioChart

- **Arquivo:** `src/components/dashboard/ritmo-diario-chart.tsx`
- **Tipo:** Client Component
- **Props:** `{ data: RitmoDiarioResponse }`
- **Visual:** LineChart com 2 séries:
 - Série 'Ideal': linha pontilhada cinza, do dia 0 a `meta_mes`
 - Série 'Realizado': linha sólida na cor do setor (ou neutra se 'todos')
 - Eixo X: dias úteis do mês
 - Eixo Y: valor BRL acumulado
 - Tooltip mostra ambos os valores e o gap

HistoricoMensualChart

- **Arquivo:** `src/components/dashboard/historico-mensual-chart.tsx`
- **Tipo:** Client Component
- **Props:** `{ data: HistoricoMensualResponse }`
- **Visual:** BarChart empilhado, 12 meses no eixo X.
 - 3 séries empilhadas: Lazer, Weddings, Corporativo (cores do `dim_setor_macro`)
 - Linha sobreposta com a meta total mensal (cinza)
 - Tooltip mostra valores por setor + total + meta

RankingVendedoresTable

- **Arquivo:** `src/components/dashboard/ranking-vendedores-table.tsx`
- **Tipo:** Server Component
- **Props:** `{ data: RankingResponse }`

- **Visual:** Tabela shadcn/ui com colunas:
 - Posição (1-10), Vendedor, Setor (badge colorido), Valor Total (BRL), Vendas (count), Ticket Médio (BRL)

RankingProdutosTable

- **Arquivo:** `src/components/dashboard/ranking-produtos-table.tsx`
- **Visual:** idem ao de vendedores, mas para produtos.

YoyChart

- **Arquivo:** `src/components/dashboard/yoy-chart.tsx`
- **Tipo:** Client Component
- **Props:** `{ data: YoyResponse }`
- **Visual:** LineChart com 2 séries:
 - Série 'Ano atual': linha sólida
 - Série 'Ano anterior': linha pontilhada mais clara
 - Tooltip mostra ambos + variação %
 - Anotação textual em meses com variação > +20% ou < -20% (opcional, se complicar pular)

Filters

- **Arquivo:** `src/components/dashboard/filters.tsx`
- **Tipo:** Client Component
- **Comportamento:** 3 dropdowns shadcn/ui (Select):
 - Ano: 2024, 2025, 2026
 - Mês: 1-12 (com nomes em português)
 - Setor: Todos, Lazer (Trips), Weddings, Corporativo
 - Mudança em qualquer filtro atualiza a URL com `router.push()` e dispara nova render server.

ThemeToggle

- **Arquivo:** `src/components/theme-toggle.tsx`
- **Implementação:** usar next-themes + shadcn/ui (componente padrão).

8. Tema Visual

8.1 Diretrizes gerais

Para esta v1, usamos o design system padrão do shadcn/ui (que oferece tema claro/escuro elegante e profissional pronto). Identidade visual da Welcome Trips será aplicada em iteração futura.

8.2 Cores dos setores macro

Setor	Cor Hex	Uso
Lazer (Trips)	#378ADD (azul)	Gauges, barras empilhadas, badges
Weddings	#BA7517 (âmbar)	Gauges, barras empilhadas, badges
Corporativo	#0F6E56 (verde-petróleo)	Gauges, barras empilhadas, badges
Total / Welcome Group	Cor neutra (cinza ou primary)	Gauge total
Meta (referência)	Vermelho/coral	Ponteiro de gauge, linha de meta

Cores dos setores ficam armazenadas em `dim_setor_macro.cor_hex` e devem ser lidas do banco (não hardcoded no frontend, exceto fallback).

8.3 Tipografia e formatação

- Fonte padrão do shadcn/ui (Inter geralmente).
- Valores monetários sempre formatados como 'R\$ 1.234.567,89' (locale pt-BR).
- Percentuais sempre com 1 casa decimal: '92,3%'.
- Datas: '04/abr', 'abr/2026', '2026' conforme contexto.
- Nomes de setores macro: usar `display_nome` do banco (ex: 'Trips' para 'Lazer').

8.4 Acessibilidade

- Contraste WCAG AA mínimo (default do shadcn/ui já atende).
- Componentes shadcn/ui já usam Radix UI por baixo, com a11y built-in.
- Não usar apenas cor para transmitir informação — sempre acompanhar com texto/ícone.

9. Missões — Divisão do Trabalho

O trabalho é dividido em 5 missões. Cada missão é um Pull Request separado, com critérios de aceite explícitos. Antes de iniciar uma missão, Claude Code propõe um plano e aguarda aprovação de Yan.

9.1 Visão geral

Missão	Foco	Branch	Estimativa
M1	Modelagem do banco	feature/m1-modelagem-banco	1-2 sessões
M2	Carga dos dados	feature/m2-carga-dados	1-2 sessões
M3	APIs (Route Handlers)	feature/m3-apis-dashboard	2-3 sessões
M4	Frontend (componentes)	feature/m4-frontend-dashboard	3-4 sessões
M5	Filtros, polimento, deploy	feature/m5-finalizacao	1-2 sessões

Sessão = ~1-2h de trabalho focado. As estimativas são para Yan trabalhando com auxílio de Claude Code. Se o trabalho fluir, pode ser mais rápido. Se travar, a missão pode ser quebrada em sub-PRs.

9.2 Missão 1 — Modelagem do banco

Objetivo: criar todas as estruturas no Supabase (schemas, tabelas, índices, views, RLS).

Entregas:

- Migrations 0001 a 0007 conforme estrutura do capítulo 4.
- Migrations rodadas no Supabase via supabase CLI (ou cole-via-painel se CLI não estiver disponível).
- dim_data populada com datas 2024-01-01 até 2030-12-31.
- dim_setor_macro populada com 3 valores fixos (Lazer, Corporativo, Weddings).
- Tipos TypeScript gerados em src/types/database.ts via supabase gen types.
- ADR 0002 'Modelagem inicial do banco' criado em docs/adr/.
- README atualizado com instrução de setup do banco.

Critério de aceite: via SQL Editor do Supabase, é possível executar 'SELECT * FROM analytics.dim_setor_macro' e ver os 3 setores. Todas as tabelas existem com colunas corretas. Tipos TS gerados sem erros.

9.3 Missão 2 — Carga dos dados

Objetivo: popular o banco com dados reais das 3 planilhas.

Entregas:

- Script supabase/seed/seed.ts implementado conforme capítulo 5.
- Script package.json atualizado: 'seed': 'tsx supabase/seed/seed.ts'
- Função SQL transform_raw_to_analytics() criada (em migration 0008).
- Carga executada com sucesso: ~31k linhas em raw, ~31k em fato_venda_item, ~19k em fato_venda.
- Metas carregadas: 36 linhas (3 setores × 12 meses × 3 anos = 108).
- Views materializadas refreshed.
- audit.ingestao_log com registros de cada execução.

Critério de aceite: queries de validação (SELECT count, SELECT SUM(valor_total) por ano) batem com os números do Anexo A.4 deste briefing.

9.4 Missão 3 — APIs (Route Handlers)

Objetivo: implementar os 6 endpoints documentados no capítulo 6.

Entregas:

- 6 Route Handlers em src/app/api/dashboard/*.
- Tipos compartilhados em src/lib/types.ts (KpisResponse, GaugesResponse, etc.).
- Validação de query params com zod em todos endpoints.
- Cliente Supabase server-side configurado em src/lib/supabase/server.ts.
- Queries SQL otimizadas (usar views materializadas onde fizer sentido).

Critério de aceite: Cada endpoint pode ser testado via curl ou via /api/<endpoint> no navegador.

Retornos têm a estrutura especificada. Performance: cada endpoint < 500ms em desenvolvimento.

9.5 Missão 4 — Frontend (componentes)

Objetivo: implementar os 8 componentes documentados no capítulo 7 + página principal.

Entregas:

- Componentes em src/components/dashboard/.
- Página em src/app/(dashboard)/page.tsx fazendo fetch paralelo das APIs.
- Layout com header e footer.
- Tema claro/escuro funcionando (next-themes + shadcn/ui).
- shadcn/ui instalado e componentes base instalados (Button, Card, Select, Table, Tooltip, etc.).
- Recharts instalado e configurado.

- Formataadores em `src/lib/formatters.ts` (`formatBRL`, `formatPct`, `formatNumber`).

Critério de aceite: rodando localmente, abrir `http://localhost:3000` mostra o dashboard completo com dados reais. Todos componentes renderizam. Tema claro/escuro alterna. Não há erros no console.

9.6 Missão 5 — Filtros, polimento, deploy

Objetivo: adicionar filtros funcionais, ajustar detalhes visuais, garantir deploy estável no Vercel.

Entregas:

- Componente Filters integrado e funcional (atualiza URL e re-renderiza).
- Loading states (`Suspense` + skeletons do `shadcn/ui`) em cada componente.
- Estados vazios tratados (mês sem dados, setor sem vendas).
- Footer com aviso 'Metas 2024 e 2025 são fictícias (decréscimo de 15% a.a. sobre 2026)'.
- Variáveis de ambiente conferidas no Vercel.
- Deploy bem-sucedido no Vercel, URL acessível.
- README com instruções de uso, setup e deploy.
- ADR 0003 'Estrutura de filtros via Search Params' criado.

Critério de aceite: Yan acessa a URL pública do Vercel pelo navegador no celular e desktop. Vê o dashboard completo. Filtros funcionam (troca ano, troca mês, troca setor — visualizações atualizam). Tema funciona. Sem erros visíveis.

10. Premissas e Decisões Registradas

10.1 Sobre dados

- **Coluna 'Intermediário' (presente em 2024 e 2025):** ignorada nesta v1, conforme decisão do Yan. Não vai para raw, não é processada.
- **Receitas negativas:** aceitas como prejuízo. Não filtradas na ingestão. Aparecerão nas somas. Visualização especial deste cenário fica para iteração futura (Onda 3 — Rentabilidade).
- **Vendas com Pagante vazio:** aceitas. Pagante = NULL no banco.
- **Datas:** apenas datas de venda. 'Data Carga' e 'Data Atualização' que viriam do RPA não existem nas planilhas — ficam para quando RPA for adaptado.

10.2 Sobre taxonomia de setores

Hierarquia oficial Macro → Setor → Micro (encontrada nos dados de 2024-2026):

Setor Macro	Setor	Setor Micro
Lazer	Lazer	Lazer
Lazer	Expedições	Expedições
Corporativo	Corporativo	Corporativo
Weddings	WedMe	WedMe
Weddings	WedMe	Hospedagem
Weddings	Weddings	Weddings
Weddings	Weddings	Hospedagem
Weddings	Weddings	Extras
Weddings	Planejamento-WED	Planejamento-WED
Weddings	Produção	Produção

Esta taxonomia é tratada como verdade nesta v1. Se mudar, atualizar as dimensões.

10.3 Sobre metas

- **Metas 2026:** valores reais fornecidos por Yan, baseados em pivot de gestores. Marcadas como fonte = 'real'.
- **Metas 2025:** calculadas como $\text{meta}_{2026} / 1.15$ para cada mês. Sazonalidade preservada. Marcadas como fonte = 'fictícia'.
- **Metas 2024:** calculadas como $\text{meta}_{2026} / 1.15^2$ ($= /1.3225$). Marcadas como fonte = 'fictícia'.
- **Aviso visual:** o dashboard exibe um aviso pequeno no rodapé quando o ano selecionado é 2024 ou 2025 sobre metas serem fictícias.

10.4 Sobre ambiente e segurança

- **Service role key:** usada APENAS no script seed local. NUNCA exposta no Vercel. NUNCA commitada no Git.
- **Anon key:** usada no frontend e nas Route Handlers de leitura. Pode ser pública.
- **RLS configurado mas permissivo:** políticas existem e estão ativas. Para esta v1, schemas analytics e app têm leitura aberta para anon. Quando login chegar, políticas evoluem.
- **Sem dados pessoais sensíveis:** nomes de vendedores e clientes são tratados como dados internos da empresa. Quando autenticação for adicionada, restringir acesso.

10.5 Decisões adiadas conscientemente

Decisões importantes que NÃO são tomadas nesta v1, e por que:

- **Versionamento de dados (slowly changing dimensions):** se um vendedor mudar de setor, ou produto for renomeado, como manter histórico? Decisão adiada para quando o problema for real.
- **Multi-tenant:** plataforma é mono-empresa por enquanto. Estrutura permite evoluir, mas não há tabela 'empresas' nesta v1.
- **Internacionalização:** tudo em pt-BR. Sem i18n. Decisão adiada para quando houver demanda real.

Anexo A — Referências e Números

A.1 Volume de dados esperado

Tabela	Linhas estimadas (após carga)
raw.vendas_excel	~31.000 (soma das 3 planilhas)
analytics.fato_venda_item	~31.000 (1:1 com raw)
analytics.fato_venda	~19.000 (vendas únicas deduplicadas)
analytics.dim_vendedor	~56
analytics.dim_pagante	~5.000
analytics.dim_produto	~98
analytics.dim_setor_macro	3
analytics.dim_setor	7
analytics.dim_setor_micro	9
analytics.dim_data	~2.557 (7 anos × 365 dias)
app.meta_setor	108 (3 setores × 12 meses × 3 anos)

A.2 Resumo financeiro por ano (validação)

Após a carga, queries de validação devem retornar exatamente estes números:

Ano	Linhas	Vendas únicas	Valor Total	Receitas	Margem %
2024	11.769	7.214	R\$ 48.628.799,78	R\$ 6.506.475,91	13,38%
2025	14.942	9.224	R\$ 60.311.487,99	R\$ 8.184.095,38	13,57%
2026 YTD	4.290	2.705	R\$ 21.999.406,37	R\$ 2.970.056,58	13,50%

A.3 Tabela completa de metas mensais

Metas 2026 (reais)

Mês	Lazer (Trips)	Corporativo	Weddings
jan	1.730.750,02	1.268.695,00	2.048.746,41
fev	2.499.737,96	1.268.694,80	2.048.746,41
mar	4.976.144,14	1.482.647,30	2.299.718,93
abr	2.367.602,37	1.450.430,10	1.762.255,41
mai	4.568.894,42	1.765.413,60	2.136.683,63
jun	3.465.682,65	1.394.935,90	1.811.492,12

Mês	Lazer (Trips)	Corporativo	Weddings
jul	2.729.013,05	1.016.725,60	1.989.324,96
ago	2.654.149,86	1.276.959,00	2.504.343,40
set	2.566.020,89	1.059.222,00	2.008.704,18
out	2.665.341,68	1.228.347,20	1.620.179,91
nov	2.507.327,88	1.579.303,90	2.083.088,39
dez	2.090.730,12	1.140.608,20	1.504.452,79
TOTAL	34.821.395,04	15.931.982,60	23.817.736,54

Metas 2025 (fictícias = 2026 / 1.15)

Calculadas dividindo cada valor de 2026 por 1.15. Total anual:

- Lazer: R\$ 30.279.473,95
- Corporativo: R\$ 13.853.897,91
- Weddings: R\$ 20.711.075,25
- Total: R\$ 64.844.447,11

Metas 2024 (fictícias = 2026 / 1.15²)

Calculadas dividindo cada valor de 2026 por 1.3225. Total anual:

- Lazer: R\$ 26.329.977,35
- Corporativo: R\$ 12.046.867,75
- Weddings: R\$ 18.009.630,65
- Total: R\$ 56.386.475,75

A.4 Análise comparativa atingimento

Cálculos derivados (úteis para validar dashboard):

Ano	Realizado	Meta	Atingimento
2024	R\$ 48,63 Mi	R\$ 56,39 Mi	86,2%
2025	R\$ 60,31 Mi	R\$ 64,84 Mi	93,0%
2026 (YTD abr)	R\$ 22,00 Mi	R\$ 16,04 Mi (jan+fev+mar+abr parcial)	—

A.5 Glossário

Termo	Definição
ADR	Architecture Decision Record
App Router	Sistema de rotas do Next.js 13+ baseado em pastas em src/app/

Termo	Definição
Backfill	Carga inicial de dados históricos
Edge Function	Função executada na infraestrutura do Supabase
RLS	Row-Level Security — permissões no nível da linha do banco
Route Handler	API endpoint do Next.js em <code>src/app/api/</code>
Server Component	Componente React renderizado no servidor (padrão Next.js)
Client Component	Componente React renderizado no cliente (com 'use client')
Service Role	Chave do Supabase com acesso total, ignora RLS — apenas server-side
Anon Key	Chave pública do Supabase, respeita RLS — pode ir pro frontend
Surrogate Key	Chave primária artificial (bigserial), independente de chave natural
YoY	Year-over-Year — comparação ano contra ano anterior

Anexo B — Como o Claude Code deve trabalhar

B.1 Início de cada missão

Para cada missão (M1 a M5), Claude Code segue esta sequência:

18. Lê a seção da missão neste briefing.
19. Cria branch a partir de main: `git checkout -b feature/<missao>`.
20. **Propõe um plano em 5-10 bullet points** e aguarda aprovação de Yan antes de codar.
21. Executa o plano em ciclos curtos.
22. Testa localmente.
23. Commit incremental com mensagens descritivas (Conventional Commits).
24. Push e abertura de Pull Request com descrição clara do que foi feito.
25. Aguarda merge antes de iniciar próxima missão.

B.2 Quando travar

- Se algo no briefing não está claro: PERGUNTAR. Não inventar.
- Se algo solicitado pelo Yan está fora de escopo: AVISAR e perguntar se é exceção ou se vai para iteração futura.
- Se algo no banco já foi feito e seria refeito: PERGUNTAR antes de executar comandos destrutivos.

B.3 Convenções de código

- **Conventional Commits:** feat:, fix:, docs:, refactor:, chore:, test:.
- **Mensagens em português:** o projeto é em pt-BR. Commits e comentários em pt-BR. Identificadores de código (variáveis, funções) em inglês.
- **Nomes de tabelas e colunas:** snake_case, em português (dim_setor_macro, valor_total). Padrão consistente.
- **Nomes de funções TypeScript:** camelCase, em inglês quando possível (formatBRL, fetchKpis).
- **Arquivos de componentes:** kebab-case (kpi-header.tsx).
- **Tipagem:** strict mode no tsconfig. Sem any, sem ts-ignore.

Encerramento

Este briefing é o ponto de partida. Decisões tomadas aqui podem ser revisadas durante a execução, desde que registradas como ADR ou comentário no PR.

A intenção não é cobrir cada detalhe perfeitamente. É dar contexto profundo o suficiente para que Claude Code execute com autonomia bem informada e Yan revise com clareza.

Sucesso na construção, Yan. Vamos para o código.